

WEEK 11

评测体系

从「凭感觉」到「可量化」——把 Copilot 的质量，做成可量化、可门禁、可监管的工程指标

01 Dataset

评测集资产

4 类样本 · 半自动生成

02 Metrics

RAGAS 量化

3 层 6 指标 + 性能成本

03 Judge

LLM 裁判

反偏差 · 校准 · 跨评

04 Gate

回归门禁

PR / Pre / Canary + A/B

05 Biz

业务指标

金融客服 4 类 · 风控 5 类

本周划重点

Week 10 我们把 Copilot 造出来了，能办事了。这一周解决一个要命的问题——你凭什么说它「办得对」？把质量从「抽测几个 case 感觉还行」，变成一个能写进 release、能卡住 PR、能交给监管看的数字。

LESSON 01 · EVAL DATASET AS ASSET

评测集资产化：从「人工抽测」到「可版本、可回归、可对抗」的评测资产

Week 10 造好了能办事的 Copilot，这一周第一步：先有一把能反复用的尺子

真实场景 · 那段每个 AI 团队都熟悉的对话

产品问：“这周效果怎么样？”工程答：“抽测了几个 case，感觉还行。”——这话在 2023 年是常态，搁 2026 年还这么说，会被客户和监管双重打回。我带团队时立过一条规矩：谁要再用“感觉还行”汇报质量，先去把评测集补上再来开会。因为“感觉”不可复现、不可对比、不可追责——它在生产里，等于没有质量保障。这一讲就讲清楚：为什么评测集必须当成“数据资产”来工程化——可版本、可回归、可对抗、可治理。

咱们这节聊 4 件事：

- 评测集相比“测试用例”，多出的 4 个工程属性：可版本 / 可回归 / 可对抗 / 可治理
- 4 类样本（happy / boundary / adversarial / multi-hop）怎么覆盖，少一类就有盲区
- 从历史日志半自动生成“问题-答案-证据”三元组，省 80% 标注工作量
- 在 OmniSupport 里搭出第一版 50-200 条的 golden set，并绑进 release manifest

DATASET = ASSET

评测集是「数据资产」，非「测试用例」——要像生产数据一样版本化、迭代、治理

传统软件的测试集：写一次、跑很久、偶尔补。AI 评测集，完全是另一种东西

AI 评测集为什么不能“做一份就不管了”？因为它脚下的三样东西，一直在动：

业务在变

- FAQ 在更新
- 政策在调整
- 用户的说法在演化
- ——昨天的标答今天就错

模型在变

- GPT-4o → GPT-5
- reranker 在升级
- 你的 prompt 在迭代
- ——同一题答案全变了

上下文在变

- Week 7 多模态
- Week 8 RAG 服务化
- Week 9 Skill
- ——系统底座一直在改

关键判断

所以评测集是一个“持续演进的资产”，不是一次性的用例。没有评测集资产化，AI 团队所有“是变好了还是变差了”的判断，全是凭感觉——而凭感觉，就是上线翻车的头号原因。这句话你记死：评测集烂，后面 metrics、judge、gate 全是空中楼阁。

MANUAL VS ASSET

人工抽测 vs 评测集资产化——这张表是判断"AI 团队成不成熟"最直接的信号

抽测 = 还在 Demo 阶段；资产化 = 进了生产阶段

维度	人工抽测	评测集资产化
样本数	5-20 个 / 凭直觉挑	50-2000 个 / 分层覆盖
版本	没有	v1 / v2 / v3 显式版本号
可重放	不可	同份数据 + 同份评测器 → 完全可重放
Bad case 沉淀	靠工程师记忆	反例库自动维护
团队协作	靠 Wiki / 群里喊	Git 跟踪 + PR review
CI 集成	不可能	PR 触发回归 + 自动门禁
长期价值	复盘时一次性用	复利资产——越用越值钱

我面试技术负责人，常问一句："你们评测集多少条、第几版、跑在 CI 里吗？" 答得上来的，基本是真生产过的；答"我们都是人工看一看"的，简历再花我也心里有数了。这张表，就是那把尺子。

FOUR CATEGORIES

评测集的 4 类样本——少一类，就有一整块评测盲区

Happy 高频典型

- 60-70% 样本
- FAQ / 常见操作
- 验证：基础能力
- 来源：产品历史日志采样

Boundary 边界长尾

- 15-20% 样本
- 同义改写 / 罕见术语 / 含糊
- 验证：泛化能力
- 来源：低频高价值场景

Adversarial 反例

- 10-15% 样本
- PII 诱导 / 越权 / 误导攻击
- 验证：安全 + 合规
- 来源：历史事故 + 红队

Multi-hop 多跳

- 5-10% 样本
- 要跨多文档 / 多步推理
- 验证：复杂推理
- 来源：真实复杂工单

关键判断

反例库（Adversarial）是整个评测集里价值最高、却 90% 团队没有的部分——因为它不来自“正常用户”，来自你踩过的坑和红队的攻击。我的判断：反例占比低于 10%，这套评测集就只测了“晴天”，一上线遇到“暴雨”必崩。

AUTO-GENERATE

从历史日志半自动生成"问题-答案-证据"三元组——把标注工作量砍到 1/5

纯人工写 50 条三元组要 8-15 小时；半自动只要 2-3 小时

```
# evals/generate.py
from anthropic import Anthropic
import json

def generate_eval_triples(docs, n=50):
    client = Anthropic()
    samples = []
    for doc in docs[:n]:
        prompt = f"""Generate a RAG eval sample.
        Given the document, create:
        1. A realistic user query (Chinese)
        2. Expected answer (must be doc-supported)
        3. Evidence chunks (cite exact spans)
        4. category: happy/boundary/adversarial
        Return JSON.
        Doc: {doc["text"]}
        Section: {doc["section_path"]} Page: {doc["page_no"]}"""
        resp = client.messages.create(
            model="claude-sonnet-4-6", max_tokens=1000,
            messages=[{"role": "user", "content": prompt}],
            response_format={"type": "json_schema", ...})
        sample = json.loads(resp.content[0].text)
        sample["source_doc"] = doc["doc_id"]
        sample["doc_version"] = doc["doc_version"]
        samples.append(sample)
    return samples
```

实战流程:

- # 1. 拉生产 RAG 日志的历史 doc
- # 2. auto-generate 100 条候选
- # 3. 人工 review 50 条 + 修正
- # 4. 手工补 10-20 条 adversarial 反例
- # 5. 写入 evals/v1.json

response_format=json_schema · Week 8 的 OpenAI Structured Outputs，保证输出能被代码直接解析

sample 带 source_doc + doc_version · 评测集和数据资产形成血缘，Week 14 治理直接消费

让 LLM 输出 category · 评测集天然按 4 类分组，省一道人工标注

关键是"半自动"这四个字 · LLM 生成 + 人工 review，绝不全自动——反例和边界，机器编不出真实感

反例那 10-20 条，必须人手来补：机器只会编"晴天题"

SAMPLE SCHEMA

一个合格评测样本的字段——绝不是只有 Q&A 两栏

字段	含义	为什么必填
sample_id	唯一标识	CI 报告关联 / Bad case 跟踪
query	用户问题	评测输入
expected_answer	标准答案（关键事实）	faithfulness 评测对照
expected_evidences	应引用的 chunk_id 列表	context_precision 评测
category	happy / boundary / adversarial	分层统计，不混在一起算
tags	业务标签（vip / cn / refund）	细分分析，定位哪类掉得狠
source_doc + doc_version	来源 + 版本	数据资产血缘
created_at + created_by	溯源	审计

少一个字段，就少一种工程能力。没有 category，你就只能看总分、看不出哪类掉；没有 doc_version，事故复盘时你连"当时用的哪版文档"都查不到。字段是后面所有分析的地基。

VERSION BINDING

评测集必须绑进 release manifest——它是第 5 类必须锁版本的对象

不同版本的评测集会算出不同的分；不锁版本，“上线前 vs 上线后”根本没法比

```
# release/manifests/rag-v2026.05.18-001.yaml
kind: RAGRelease
spec:
  index:    {...} # Week 4/7
  prompt:  {...} # Week 8
  model:    {...} # Week 8
  skill_pack: {...} # Week 9
  eval:     # Week 11 ← 第 5 类必绑
    dataset:
      id: rag_qa_golden_v2
      version: 2.3.0
      sample_count: 150
      categories: {happy: 95, boundary: 35,
                    adversarial: 20}
      generated_from:
        - {source: production_logs, count: 110}
        - {source: bad_case_archive, count: 20}
        - {source: red_team, count: 20}
      digest: sha256:abcdef...
    baseline_metrics: # 回归门禁的对比锚点
      faithfulness: 0.892
      answer_relevance: 0.913
      context_precision: 0.870
      adversarial_pass_rate: 0.95
```

eval.dataset 段 · 把“这次发布用了哪份评测集”显式钉死，release_id 直接对应一份评测集版本

categories 拆分 · 一眼看出反例够不够：20/150 ≈ 13%，合格

generated_from · 记清来源，告诉你这份集能不能继续扩

digest 内容指纹 · 任何样本改动 hash 就变，复盘时精确定位“是不是评测集动过”

baseline_metrics · 下一版上线时的“对比基线”——这就是 Lesson 4 回归门禁的硬约束

INDUSTRY 2026

评测集资产化，早已是行业标准基础设施——别再当"内部小工具"

2026 年：开源生成 + 平台托管，两条腿都成熟了

工具 / 实践	能力	怎么用
RAGAS TestsetGenerator	从知识库自动合成 RAG 评测集	半自动生成的开源首选，"先用它探索"
DeepEval Synthesizer（2026 主流）	pytest 风格 + 数据合成 + CI 集成	"用 RAGAS 探索，用 DeepEval 卡门禁"
OpenAI Evals / Anthropic Eval Recipes	版本化 eval set + step-by-step 模板	别从零造，套官方 recipe
Hugging Face Datasets	数百个公开 eval set	通用能力直接复用，省采样

记死

2026 年的事实 → 评测集已是 Agent / RAG 项目的标准基础设施。落地路径很清楚：业务垂直集自己半自动生成（RAGAS 探索 + DeepEval 固化进 CI），通用能力直接拿 HF 公开集。把精力花在"反例库"上，那才是你的护城河。

LESSON 02 · RAGAS METRICS

RAGAS 多维量化：把「答得好不好」变成「F=0.892 / AR=0.913 / CP=0.870」

上一讲做完了评测集这把尺子，这一讲给尺子刻上刻度

真实场景 · "答对率 80%，还行吧？"

我见过太多团队，质量就报一个数："答对率 80%。" 我每次都追一句：哪 20% 错的？是模型编的、是答非所问、还是压根没检索到？十有八九答不上来。问题就在这——"对/错" 这一个维度，把三种完全不同的病混成了一锅。答案看着对，可能是模型自己编的（没引用证据）；看着全，可能引了一堆不相关的 chunk；答得完美，可能检索压根没找到关键资料。RAGAS 的精妙，就是把这一个"对错"，拆成能分别下药的多个工程维度。

咱们这节聊 4 件事：

- RAG 评测为什么必须拆成"检索 / 生成 / 整体"三层、6+ 个维度
- RAGAS 三大核心指标（Faithfulness / Answer Relevance / Context Precision）各测什么
- 指标低了别慌——按"诊断→治疗"对照表精准下药，不要全部重做
- RAGAS 之外还要测的 latency / cost / safety——5 类指标合起来才是健康仪表盘

THREE LAYERS, SIX METRICS

RAG 评测不能用一个分数——必须拆成「检索 / 生成 / 整体」三层

只看“答对率”是新手做法；看 6 个维度，才知道到底是哪一层在拖后腿

检索层

- context_precision
- context_recall
- 问：资料找对了吗 / 找全了吗
- 低 → rerank / chunk / hybrid

生成层

- faithfulness
- answer_relevance
- 问：有没有编 / 答没答到点
- 低 → prompt / query 改写

整体层

- answer_correctness
- semantic_similarity
- 问：综合对不对 / 跑没跑偏
- 低 → 回到上面拆根因

关键判断

记住这个三层拆法，你看任何 RAG 项目都能直击要害：先看检索层——资料都没找对，生成再强也是巧妇难为；再看生成层——资料对了还编，那是 prompt 的锅。把“对错”拆成 6 个维度，“哪一层在拖后腿”立刻一目了然。

THREE CORE METRICS

RAGAS 三大核心指标——每一个解决一个不同的工程问题

阈值是工程经验值，不是绝对真理，按你的业务风险上调

Faithfulness 忠实度

- 生成层
- 答案每个 claim 是否都来自上下文
- 低 → LLM 在编造
- 阈值 > 0.85 (金融可拉到 0.92)

Answer Relevance 相关性

- 生成层
- 答案和"提问意图"的相关程度
- 低 → 答非所问 / 偏题
- 阈值 > 0.80

Context Precision 精准度

- 检索层
- 召回的 chunk 里"真相关"的占比
- 低 → 召回了太多噪声
- 阈值 > 0.75

关键判断

Faithfulness 比 Answer Relevance 更重要——但很多团队反过来。因为"答非所问"用户一眼看穿，而"编得有理有据"最坑：它骗得过用户，却骗不过监管。金融场景，Faithfulness 是第一红线。

METRIC SEMANTICS

RAGAS 6 个指标完整对照——看完就知道每个分数背后到底在测什么

这 6 个合起来 = RAG 系统的体检表，少跑一个就有盲区

指标	层级	怎么算	低值意味着
Faithfulness	生成	逐句 claim 能否在 context 找到	LLM 在编造
Answer Relevance	生成	答案 ↔ 问题意图的语义匹配	答非所问
Context Precision	检索	召回 chunk 中相关的占比	召回噪声多
Context Recall	检索	该召回的 chunk 实际召回比例	漏召关键资料
Answer Correctness	整体	F + AR + 与标答对照	综合差
Semantic Similarity	整体	与标答的 embedding 距离	语义跑偏

检索层两个（CP/CR）是一对：CP 低=召回太脏，CR 低=召回太漏，方向相反，治法也相反。很多人只跑 CP 不跑 CR——结果"召回很干净"但其实漏了一半关键资料，分数好看、答案残缺。两个一起看才不骗自己。

RAGAS REAL CODE

OmniSupport 项目里跑通 RAGAS——一次 forward 出 6 个指标

直接消费 Lesson 1 的评测集，同一个 source of truth

```
# evals/run_ragas.py
from ragas import evaluate
from ragas.metrics import (
    faithfulness, answer_relevancy,
    context_precision, context_recall,
    answer_correctness, answer_similarity)
from datasets import Dataset
import json

samples = json.load(open("evals/v2.json")) # L1 产出
eval_data = []
for s in samples:
    resp = rag_query(s["query"], tenant="eval")
    eval_data.append({
        "question": s["query"],
        "answer": resp["answer"],
        "contexts": [e["text"] for e in resp["evidences"]],
        "ground_truth": s["expected_answer"],
    })
ds = Dataset.from_list(eval_data)

result = evaluate(ds, metrics=[
    faithfulness, answer_relevancy,
    context_precision, context_recall,
    answer_correctness, answer_similarity],
    llm=judge_llm, # ← 用"不同family"模型当 judge
    embeddings=judge_emb) # bge-large 做 semantic

result.to_pandas().to_csv("eval-report-v2.csv")
```

samples 直接读 Lesson 1 的评测集 · 同一个 source of truth, 不另起炉灶

eval_data 字段对齐 RAGAS schema ·
question/answer/contexts/ground_truth

evaluate() 一次跑 6 指标 · 避免对 RAG 服务多次重复调用，省钱省时

judge_llm 这行是命门 · 用 GPT-5 评 GPT-5 写的答案 = 自评，bias 极大；生产一定用不同 family 模型 cross-evaluate (Lesson 3 细讲)

to_pandas() 直接接 Grafana / dashboard · 评测结果要能被看见

judge 一轮全量 (150 样本 × 6 指标) API 成本约 \$1-3 · 便宜得很，别为省这点钱跳过评测

LOW METRIC → ACTION

指标低了具体改什么——RAG 项目最高频用到的"诊断→治疗"对照表

指标低不要慌、更不要全部推倒重做，按这张表精准下药

指标低	病根	应对动作	对应周次
Faithfulness < 0.85	LLM 编造 / 引用错位	加严 prompt + Structured Outputs	Week 8 L3/L4
Answer Relevance < 0.80	答非所问 / 偏离意图	Query 改写 / Multi-Query	Week 8 L1
Context Precision < 0.75	召回噪声太多	加 reranker + 调阈值	Week 8 L2
Context Recall < 0.75	关键资料漏召	换 chunk 策略 / 上 hybrid	Week 7 L2 / 8 L1
Answer Correctness < 0.70	综合错	看上面拆根因，别瞎改整体	回到上面

关键判断

这张表我自己天天用。新手一看总分低就重做整条链路，浪费一周还不一定好；老手先拆 6 维、定位到具体那一层，改一个 reranker、调一行 prompt，半天解决。诊断清楚了，治疗就是精准的——这就是多维评测的全部价值。

BEYOND RAGAS

RAGAS 之外，3 个最常被忽视、却能要命的指标

RAGAS 测的是"质量"; 生产 RAG 还得测"性能 / 成本 / 安全"

指标	为什么必须测	怎么测
Latency P99	答得慢，用户直接跑光	OpenTelemetry trace 统计（Week 12）
Cost per Query	上线后费用失控的头号元凶	token usage × 模型单价汇总
Refusal Rate	该拒的有没有拒（反例库验证）	反例库 + 自动检查
Citation Accuracy	引用是否真的支撑 claim	Anthropic Citations API + 人工抽样
HITL Trigger Rate	人工介入触发得过多 / 过少	从 Week 10 audit 统计

Cost per Query 这个，我吃过亏：一个 demo 阶段没人管成本的 RAG，上线后每月 API 账单冲到 \$10 万+，一查是每次召回 top_k=20、还套了三轮 LLM 重排。质量分再漂亮，成本失控一样会被叫停。质量、性能、成本、安全——四个轮子，缺一个车就翻。

EVAL DASHBOARD

生产级 RAG 评测看板——5 类指标统一展示

Copilot 的健康仪表盘：每周复盘、每月对基线、每次发布对上一版



关键判断

这 5 类合起来，才是 Copilot 真正的“健康仪表盘”。单看 Quality 那一格，就像只量体温不量血压——看着没烧，可能血压早爆了。把质量从“主观感觉”变成“5 类指标的可量化趋势”，你才第一次真正“看得见”系统的健康。这套看板，Week 12 会接到全链路可观测里去。

INDUSTRY 2026

RAG 评测工具生态——2026 年别再"自己造评测框架"

路线已经清晰：开源探索 + 平台门禁，两条腿走

工具	定位	2026 怎么选
RAGAS	RAG 4 指标 / 无需标注 / 轻量	"快速探索"首选，像用 pandas 做分析
DeepEval	pytest 风格 / TDD / CI 集成	"卡门禁"首选——explore with RAGAS, gate with DeepEval
Braintrust	eval + CI/CD 门禁 / PR 自动评分	小团队 \$0（免费档 1M spans/月）就能上 eval gate
Langfuse / Phoenix / LangSmith / TruLens	trace + eval 一体 / 自托管 or 平台	不想自建就上：自托管选 Langfuse

记死

2026 年的共识打法 → 用 RAGAS 快速探索指标，用 DeepEval 把评测固化成 pytest 用例进 CI，用 Braintrust / Langfuse 做 trace + 门禁 + PR 评分。这套组合拳，比你自己造一个评测框架，强十倍、省三个月。

PITFALLS

RAG 评测最常见的 5 个反模式——尤其第 2 个最隐蔽

这些坑大多数团队都踩过，认出来就躲过了

反模式	具体表现	后果	正确做法
只看一个分数	"答对率 80% 还行"	没法定位问题	6+ 维度拆解
同模型自评	GPT-5 判 GPT-5 答的对错	systematic bias	cross-evaluate
评测集太小	< 30 样本就 ship	指标方差大	至少 100 样本
指标无基线	"今天 0.89" (然后呢?)	不知变好变差	永远和上一版对比
没有反例	只测 happy path	上线被反例打爆	Lesson 1 的 adversarial

踩坑提醒

这 5 个坑里，"同模型自评"最隐蔽——因为它的分数往往特别好看，让你飘飘然，可生产里掉得最快。原因很简单：模型偏爱自己风格的输出，GPT 给 GPT 打分天然偏高。我的硬规矩：任何要拿去做决策的评测，判分的模型必须和被判的模型不是一家——这是 Lesson 3 的主题。



LESSON 03 · LLM AS JUDGE

LLM 当裁判：把人工抽测成本砍到 1/100——但小心 4 类系统性偏差

上一讲那些语义指标，其实都是 LLM 在打分；这一讲把“裁判”本身管起来

真实场景 · 第一次用 LLM-as-Judge 的人都这么干

做法很自然：把答案 + 标答丢给 GPT，让它打 1-5 分。上线后你会发现一堆怪事：同一份答案，今天打 4 分、明天打 3 分；答案排在 prompt 前面的，分数系统性偏高；答案写得长的得分高、短的得分低，哪怕内容一样好；更绝的是——judge 模型偏爱给“和自己风格像”的答案打高分。我跟你讲，这不是 prompt 没写好，是 LLM 这种架构的固有毛病。用 LLM 当裁判，绝不是“换个 prompt”那么简单——它必须配套校准、反偏差、cross-evaluate。

咱们这节聊 4 件事：

- LLM-as-Judge 的 4 类系统性偏差：位置 / 长度 / 自我偏好 / 拒绝
- 3 种 Judge 模式（Pairwise / Single Score / Reference-based）按场景怎么选
- 写一份生产级 judge prompt：评分锚点 + 反偏差约束 + calibration example
- 用 100 条人工金标准校准—— $\kappa < 0.6$ 的 judge 不能拿去做生产决策



NOT JUST A PROMPT

LLM-as-Judge 是一套必须校准、反偏差、cross-evaluate 的工程实践

裁判本身不可信，你评出来的所有分数就都不可信

一个"裸奔"的 LLM judge，会同时犯这四个错，而且你不校准根本发现不了：

打分飘

- 同份答案
- 不同时间 ± 1 分
- 没锚点 $\rightarrow$ 自由发挥

位置偏

- 排前面的
- 系统性得分高
- 注意力对位置敏感

长度偏

- 长答案
- 天然得分高
- 误把"长"当"详细"

自我偏好

- 给自己家模型
- 打高分
- 训练风格相似

关键判断

LLM-as-Judge 必须四样配齐：明确的评分锚点、反偏差约束、人工校准、cross-evaluate——缺一个，评测结果就不可信。这不是"高级玩法"，是底线。一个没校准过的 judge 打出来的 0.9，和算命没区别——而你拿它去卡上线，等于拿算命决定发不发版。

FOUR SYSTEMATIC BIASES

LLM-as-Judge 的 4 类系统性偏差——以及每一类的工程消除法

注意：不是"换个更强的模型"就能解决，是架构固有特性，必须用工程手段对抗

偏差	具体表现	为什么发生	消除方法
位置偏差	A/B 顺序换一下分数就变	LLM 注意力对位置敏感	两个顺序各跑一次取平均
长度偏差	长答案分数系统性偏高	误以为"长 = 详细"	prompt 显式声明"长度不计分"
自我偏好	GPT 给 GPT 打高分	训练数据风格相似	永远 cross-evaluate
拒绝偏差	说"我不确定"反而比错答得分高	LLM 偏好谦逊回答	prompt 显式定义"不确定"的扣分规则

踩坑提醒

"换 GPT-5、换 Claude Opus 4.8 是不是就没偏差了？"——这是学员最常问的，答案是不行。模型越强，偏差更隐蔽、更难发现，但它还在。位置偏差、自我偏好这些是 Transformer 这套架构带来的，不是能力问题。所以别指望"等更好的模型"，老老实实上工程手段：换序取平均 + cross-evaluate + 人工校准。

THREE JUDGE PATTERNS

3 种 LLM Judge 模式——按场景选，生产默认 Reference-based

没有最好的模式，只有最适配你场景的模式

Pairwise 两两对比

- A vs B 谁更好
- 比单分准、和人类一致性 > 80%
- 适合：A/B 测试 / 模型对比
- 坑：位置偏差最严重，必须换序

Single Score 单条打分

- 给一条答案打 1-5 分
- 简单、可累积成趋势
- 适合：批量评测 / 历史趋势
- 坑：分数飘 ± 1 ，必须锚点校准

Reference-based 对标答

- 答案 vs 标准答案 对照
- 最稳定、最可解释
- 适合：有 ground truth 的场景
- 坑：标答构建成本高（但值）

关键判断

生产推荐默认 Reference-based——因为有标答兜底，judge 的发挥空间最小、最稳。Pairwise 准但只给相对结果（适合选型），Single 灵活但飘（适合看趋势）。一句话：能构建标答就别裸评。



GOOD JUDGE PROMPT

OmniSupport 里 faithfulness 的生产级 judge prompt

一份好 judge prompt 和差的, 同一批数据能差出 30% 的一致性

```
# evals/judges/faithfulness.j2
```

You are evaluating whether an AI response is "faithful" to the provided context (RAG-style). Faithful = EVERY factual claim must be directly supported by the context chunks.

Scoring criteria: # ← 4 档锚点

- 1.0: All claims supported by context
- 0.7: Most supported, 1-2 minor extrapolations
- 0.4: Half supported, half made-up
- 0.0: Most claims fabricated

IMPORTANT: # ← 直接对抗偏差

- Do NOT penalize answer length (long != better)
- "I don't know" = 0.5 if context insufficient
- Rephrased citations still count as supported

Example (calibration anchor):

Context: "重置密码 3 步: 设置→密码→输入新密码"

Answer: "3 步即可: 设置 → 密码 → 输入新密码"

Score: 1.0 // 完全 faithful

Answer: "重置密码 5 步: 先登录、再进入设置..."

Score: 0.4 // 编造了"5 步"和"登录"

Now evaluate:

Context: {{ context }} Answer: {{ answer }}

Output JSON: {"score":float,"reason":"...",
"unsupported_claims":[...]}

Scoring criteria 给 4 档锚点 · LLM 不再在 0-1 之间"自由发挥", 分数标尺被钉死

IMPORTANT 段直接对抗偏差 · 长度偏差、拒绝偏差, 都在 prompt 里强制约束

Example 是 calibration anchor · 给 LLM 看具体对错示例, 把刻度校准到位

Output JSON · Week 8 的 Structured Outputs, unsupported_claims 能被代码消费、回流反例库

这份 prompt 改几个字, 效果能差 30% · prompt 才是 LLM-as-Judge 的真正灵魂

HUMAN CALIBRATION

LLM judge 像一把"自动卷尺"——但出厂刻度不一定准，必须用人工标注校准

标 100 条人工金标准，跑 judge 看一致性：> 80% 可用，< 70% 重写

一致性指标	怎么算	建议阈值	不达标怎么办
Cohen's κ	排除随机一致后的真实一致度	> 0.6 = good	改写 judge prompt
Pearson r	连续分数的相关系数	> 0.75	加 calibration anchor
MAE	平均绝对误差 (0-1 scale)	< 0.15	换更强的 judge 模型
Top-K 一致	人工 top-K 与 LLM top-K 重合度	> 80%	看具体偏差类型对症下药

$\kappa > 0.6$ 是 Krippendorff 那条通用阈值——低于它，judge 和人的判断分歧大到不能用来做决策。这 100 条金标准只需标一次（重大场景变更再重标），却是你整套自动评测的"信任之锚"。我的规矩：judge prompt 一改，必须重跑校准——因为改 prompt 等于换了个裁判。



CALIBRATION CODE

用 100 条人工金标准校准 LLM judge——4 个角度算一致性

校准报告随 judge prompt 一起 git 跟踪，写进 release manifest

```
# evals/calibrate.py
from sklearn.metrics import cohen_kappa_score
from scipy.stats import pearsonr
import numpy as np, json

def calibrate_judge(human_set, judge_fn):
    samples = json.load(open(human_set))
    human, judge = [], []
    for s in samples:
        human.append(s["human_score"]) # 0/0.5/1
        judge.append(judge_fn(
            context=s["context"],
            answer=s["answer"])[ "score"])
    human, judge = np.array(human), np.array(judge)
    # 1. Cohen's  $\kappa$  (离散化分箱)
    kappa = cohen_kappa_score(
        np.round(human*2), np.round(judge*2))
    # 2. Pearson r 3. MAE
    r, _ = pearsonr(human, judge)
    mae = np.mean(np.abs(human - judge))
    # 4. Top-K 一致
    k = 20
    h_top = set(np.argsort(human)[-k:])
    j_top = set(np.argsort(judge)[-k:])
    return {"cohen_kappa": kappa, "pearson_r": r,
            "mae": mae,
            "top_k_overlap": len(h_top & j_top)/k,
            "trust_level": "high" if kappa>0.6 else "low"}
#  $\kappa$ <0.6 → 重写 judge prompt → 重新校准
```

4 个指标从不同角度看一致性 · 单一指标会误导 (κ 看分类一致、r 看趋势、MAE 看绝对误差、Top-K 看排序)

100 条金标准做一次、复用极强 · 重大场景变更时才重做

trust_level 写进 release manifest · Week 14 治理直接消费——judge 不可信，这次发布就该卡住

校准报告随 judge prompt git 跟踪 · 改 prompt = 改裁判，必须重新校准，否则等于偷换裁判

这套东西不写，你的所有“自动评测”都是建在沙子上

CROSS-EVALUATE

Cross-Evaluate：用至少 2 个不同 family 的模型互相打分，取共识

单模型 judge 有自我偏好——尤其当 RAG 本身也用同一模型生成时

策略	具体做法	成本	准确度
Single Judge	只用一个 LLM 评判	1x	低（有 bias）
Pair Judge + Average	GPT-5 + Claude 各跑取平均	2x	中高（消除大部分 bias）
Pair Judge + Disagree Flag	两家分歧 > 0.3 标记人工二审	2x	高
Multi-Judge Voting	GPT-5 + Claude + Gemini/DeepSeek R1 投票	3-5x	最高（合规 / 资金场景）

关键判断

生产推荐：常规评测用"Pair Judge + Average"（如 GPT-5 + Claude Sonnet 4.6，预算紧可用 DeepSeek R1 当便宜的第二裁判）——成本翻一倍，但把系统性偏差基本消干净，值。高风险场景（合规话术、资金动作）上 Multi-Judge Voting，贵一点但出事代价更贵。记住那个最隐蔽的陷阱：如果你的 RAG 用 GPT-5 生成、又用 GPT-5 当裁判，那分数高得离谱也别信——那是自己夸自己。

INDUSTRY 2026

LLM-as-Judge 早不是"能不能用", 是"怎么用对"的工程问题

2026 年: pairwise + cross-evaluate + 校准锚点, 已是公认底座

来源	关键结论	工程含义
Anthropic Eval 实践	明确推荐 Pairwise + Cross-Evaluate, 强调避免自评	裁判和选手不能是一家
OpenAI gpt-judge 公开数据	LLM judge vs 人类 $\kappa \approx 0.62-0.7$	"可用但必须校准", 别裸信
LMarena (原 LMSYS Arena)	pairwise 模式下与人类一致性 > 80%	pairwise 是事实工业标准
DeepEval / Braintrust 内置 judge	反偏差 + 校准 anchor 做成模板	别自己从零写 judge 框架

记死

2026 年的事实 → LLM-as-Judge 是整个自动评测体系的发动机, 但发动机得先校准。GPT-judge 和人类的一致性 κ 也就 0.65 上下——"可用、但必须配人工校准 + cross-evaluate"。谁跳过校准直接拿 judge 分数做上线决策, 谁就是在用没校准的仪器做手术。

LESSON 04 · REGRESSION GATE

回归门禁与 A/B：让评测从"周报数字"变成"PR 能不能合并"的硬约束

前三节建好了评测集/指标/裁判；这一讲让它们真正"咬合"进生产

真实场景 · 评测结果只活在周会 PPT 上

我见过一个团队，评测做得挺像样：RAGAS 跑着、看板挂着，每周五给老板汇报一页漂亮的趋势图。可你猜怎么着——线上质量该退化还退化。为什么？因为那份评测，从来没拦住过任何一个 PR。工程师改完 prompt 直接合并上线，评测周一才跑，发现退化时，客户已经骂了一个周末了。我跟你讲：不阻断的评测，等于没有评测——它只是一份"美化过的日志"。评测真正的力量，是变成"PR 能不能 merge"的那道闸。

咱们这节聊 4 件事：

- 评测体系成熟度 5 级（L0-L4），大多数团队卡在 L1-L2，升 L3 工程量不大但效果质变
- 3 层门禁：PR Gate / Pre-Release Gate / Canary Gate，各拦一类问题
- A/B 测试必须"事前算样本量、事后两道检验都过（t-test + 非参数）"，不是看哪个分高就上
- 在 OmniSupport 里跑通"自动评测 → 退化拦截 → 灰度发布"完整链

NO BLOCK, NO EVAL

评测必须接 CI/CD——不阻断的评测，等于没有评测

$$\text{评测} + \text{CI/CD} = 1 + 1 > 10$$

只有当评测变成"硬约束"，工程师才会真的在乎它。三道闸，拦三类问题：

PR 闸

- 评测变成
- "能不能合并"
- → 工程师才真在乎

上线闸

- 评测变成
- "能不能上线"的最后一道
- → 退化不漏给客户

灰度闸

- 评测变成
- "灰度能不能升档"的自动决策
- → 风险锁在 5% 流量内

关键判断

评测结果出在 Slack、Confluence、周会上——它就是个"美化的日志"，不会让系统真的变好。这就是"评测驱动开发"（Eval-Driven Development）的核心：把评测从"事后看的数字"，做成"事中卡的闸门"。一句话——能 block 的评测才有牙齿，不能 block 的评测只是装饰。

EVAL MATURITY

评测体系成熟度的 5 个等级——对照一下，你的团队在哪一档

大多数团队卡在 L1-L2；升到 L3 的工程量不大，但效果是质变

等级	具体表现	能拦住什么	能避免什么
L0 · 无评测	凭感觉抽测	什么都拦不住	——
L1 · 周报评测	每周跑一次给老板看	只能看趋势	退化 1 周后才发现
L2 · PR 评测（无门禁）	PR 时自动跑但不阻断	看到问题靠人推动	靠人记得检查
L3 · PR 评测 + 门禁	退化 > 阈值 = 不能合并	90% 退化	生产事故
L4 · PR + 上线 + Canary	+ Canary 自动升档/回滚	99% 退化	客户感知到问题

我评估一个 AI 团队的工程成熟度，就看这一张表落在哪。L1（周报）和 L2（PR 跑但不拦）是最大的两个坑——看着有评测，其实没牙。从 L2 升到 L3，往往就是加一个 exit 1、几十行 CI 脚本的事，但它把"质量保障"从"靠人自觉"变成"系统强制"。这一步，是 Demo 团队和生产团队的分水岭。

THREE GATE LEVELS

评测门禁的 3 个层级——一个变更要过哪几道闸

越往后，样本越多、决策越慢、拦的问题越接近真实流量

PR Gate

- 改 prompt/索引/Skill 触发
- 跑 golden set 50-200 条
- 退化 > 2% → block
- 5-10 分钟内决策
- GitHub Actions / GitLab CI

Pre-Release Gate

- 合到 main 后跑全量
- 全指标 + 反例库 1000+ 条
- 退化 > 1% → 阻塞发布
- 20-30 分钟决策
- 上线前最后一道把关

Canary Gate

- 上线后 5% 流量看实时
- quality + latency + cost
- 退化触发自动回滚
- 动态观察 30-60 分钟
- 接 Week 8 release manifest

关键判断

三道闸缺一不可：PR Gate 拦"改坏了"，Pre-Release 拦"全量才暴露的退化"，Canary 拦"只有真实用户流量才能发现的问题"。PR Gate 对反例库要"零退化"——安全类指标比性能类严。

GITHUB ACTIONS GATE

OmniSupport 里的 PR 评测门禁——退化超阈值，PR 直接 block

把评测从“周报”升级成“PR 能不能合并”，就靠这套 CI

```
# .github/workflows/rag-eval-gate.yml
name: RAG Eval Gate
on:
  pull_request:
    paths: ["prompts/**", "tools/**",
            "pipelines/chunk/**",
            "pipelines/retrieval/**"]
jobs:
  eval-gate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run eval on golden set
        run: python evals/run_ragas.py \
          --eval-set evals/v2.3.0.json \
          --baseline release/baseline-metrics.json
      - name: Check regression thresholds
        run: python evals/check_regression.py \
          --max-drop faithfulness=0.02 \
          --max-drop context_precision=0.03 \
          --no-drop adversarial_pass_rate
        # 任何超阈值 → exit 1 → block PR
      - name: Comment results on PR
        uses: actions/github-script@v6
      - name: Block on regression
        if: failure()
        run: exit 1
```

paths 触发条件 · 只有改了
prompt/tools/chunk/retrieval 才跑，不浪费 CI 资源

--max-drop 参数化阈值 · 每个指标单独设，写在命令行不
硬编码进代码

--no-drop adversarial_pass_rate · 反例库一条都不许退
——安全类比性能类更严

Comment on PR · 把报告贴在 PR 里，review 时一眼看到
影响，不用翻 CI 日志

failure → exit 1 · 这一行是灵魂——评测从此有了“否决权”。
2026 年 Braintrust 把这套做成开箱即用的 GitHub
Action

A/B TEST

A/B 测试：评测告诉你"绝对分"，但上线决策要的是"新旧哪个更好"

RAG 的 A/B 有特别的坑：样本量、统计检验、分布非正态

对比维度	具体例子	看哪些指标	建议样本量
Chunk 策略	fixed-512 vs semantic chunking	context_precision / recall	300+
Rerank 模型	bge-reranker vs Cohere Rerank	CP + latency + cost	500+
Prompt 版本	v3.2.0 vs v3.2.1	F / AR / hallucination	200+
LLM 模型	GPT-5 vs Claude Sonnet 4.6	6 指标 + 成本	1000+

样本量不是"越多越好"，是"够用就好"——按效应大小（effect size）事前算。效应越小，要的样本越多：大效应（>10% 改善）80 条够，中效应（5%）要 200 条，小效应（2%）得 800 条。别上来就"跑 30 个看看"——30 个样本的方差，能让一个真退化看着像进步。下一页给公式。

A/B SAMPLE SIZE

A/B 样本量计算 + 双重统计检验——把"看图说话"变成"自动决策"

RAG 分数常常非正态, 必须 *t-test* + *Mann-Whitney* 双重验证

```
# evals/ab_test.py
from scipy.stats import ttest_ind, mannwhitneyu
from statsmodels.stats.power import TTestIndPower
import numpy as np

def required_sample_size(effect=0.05, alpha=0.05,
                        power=0.8):
    """事前算: 要检测出 effect 改善, 需多少样本"""
    return int(np.ceil(TTestIndPower().solve_power(
        effect_size=effect, alpha=alpha, power=power)))
# 大效应>10%: ~80 / 中效应5%: ~200 / 小效应2%: ~800

def ab_compare(a, b):
    diff = np.mean(b) - np.mean(a)
    _, p1 = ttest_ind(a, b)      # 正态假设
    _, p2 = mannwhitneyu(a, b)  # 非参数, 更鲁棒
    sig = (p1 < 0.05) and (p2 < 0.05) # 双重通过才算数
    return {"diff": diff, "p_ttest": p1,
            "p_mwu": p2, "significant": sig,
            "recommendation":
                "ship_B" if sig and diff > 0
                else "keep_A" if sig
                else "need_more_data"}
```

required_sample_size 事前算 · 避免跑了几百条才发现"样本不够、白跑"

t-test + Mann-Whitney 双重判断 · RAG 分数分布常非正态, 必须配非参数检验交叉验证

significant 要两个 p 都 < 0.05 · 比单一检验稳得多

recommendation 直接给决策 · 不是"看图说话", 是 ship_B / keep_A / need_more_data 三选一

CI 里直接调这套 · A/B 报告 + 决策建议一体化, 把"拍脑袋上线"挡在门外

CANARY GATE

Canary 灰度：上线后的最后一道门——让 5% 流量先跑，好就升档、差就秒回滚

有些问题只有真实用户流量才暴露，PR/Pre-Release 都拦不住



关键判断

Canary 是 Week 8 release manifest + Week 10 trace audit + Week 11 在线评测，三者合起来才撑得住的"自动门"。它的价值在"自动"两个字——退化触发"自动回滚"，而不是"指标差了等人工处理"。我踩过的坑：Canary 配了告警没配自动回滚，结果半夜 3 点指标崩了、没人响应，等早上发现已经放了 6 小时坏版本。自动回滚那行代码，比任何告警都值钱。

INDUSTRY 2026

Eval-Driven Development 已是 2026 的工程范式，更是合规要求

没有 eval gate 的 AI 团队，要么没规模化，要么走在事故路上

来源	能力 / 要求	工程含义
Anthropic 内部 SOP	每次 模型/prompt/工具 改动 → 触发 eval → 不过不上	EDD 已是大厂日常
Braintrust (2026 EDD 标杆)	GitHub Action 每个 PR 自动跑评测 + 贴评分	一行 Action 就上 eval gate
LangSmith / Langfuse / Phoenix	CI 集成 + trace + 自托管	不想自建，平台直接接 CI
EU AI Act (2026.5 Omnibus)	高风险 AI 重大变更前必须有评测报告	高风险大限延至 2027.12.02，但评测义务不变

记死

2026 年的事实 → 自动评测已经从"工程提效"，升级成"合规必备"。EU AI Act 虽把高风险系统的大限延到了 2027 年底，但"重大变更前必须有评测报告"这条没松。换句话说：eval gate 现在不做，将来不是"要不要补"的问题，是"罚不罚款、能不能上线"的问题。

PITFALLS

评测门禁的 5 个常见反模式——做对了，系统稳定性能差 10 倍

反模式	具体表现	后果	正确做法
warn 不 block	PR 有 warning 但能合并	90% 人忽略 → 退化上线	硬 block + 显示具体退化
阈值拍脑袋	"退化 5% 内算正常"	过松漏退化 / 过紧老报警	基于历史 P95 trend 定
只看 mean	"mean -2% 还行"	某 segment 暴跌没看到	看 percentile + 分 segment
A/B 不算样本量	"跑 30 个差不多"	统计不显著瞎决策	事前算样本量
Canary 不自动回滚	"指标差再人工处理"	半夜 3 点没人响应	阈值触发自动回滚

踩坑提醒

第 1 个"warn 不 block"最普遍也最坑——团队觉得"先 warn 温柔点、别挡住开发"，结果 warning 没人看，退化照样上线。我的硬规矩：能 block 的就别 warn。温柔的门禁等于没门禁。要么不设，要么就给它牙齿——中间态"看得到拦不住"是最差的，既花了 CI 成本，又没拦住事故。

LESSON 05 · BUSINESS METRICS

业务评测指标：让评测从"系统好不好"升级为"业务有没有变好"

前 4 节做了技术指标体系；最后一讲，把它翻译成老板和监管听得懂的话

真实场景 · CEO 不在乎你的 $F=0.892$

我陪团队向 CEO 汇报过一次 AI 客服项目，工程同学一上来就是 "Faithfulness 0.89、Context Precision 0.87"。CEO 听完一脸茫然，问了三个问题：客服自助解决率涨了没？客户满意度上来没？合规拦截率达标没？——全答不上来。那一刻我就明白：技术指标再漂亮，翻译不成业务结果，在老板眼里就等于没做。技术指标是手段，不是目的。这一讲讲清楚：技术指标 → 业务指标中间那条"漏斗"怎么 wire，以金融客服域 + 风控域两个典型场景为例，把工程分数翻译成业务价值。

咱们这节聊 4 件事：

- 为什么 RAGAS 分高、业务指标却不动——中间缺了一条"业务漏斗"

- 金融客服 Copilot 的 4 类业务 SLO（自助 / 效率 / 质量 / 合规），缺一类无法上线

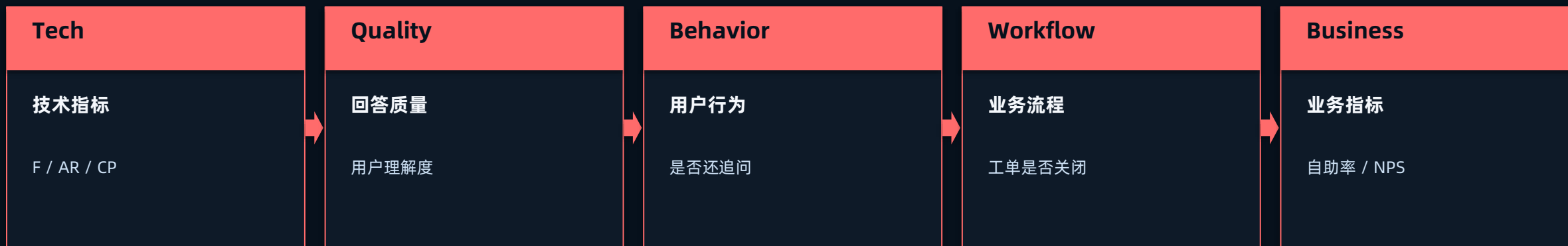
- 风控 AI 的评测完全不同——AUC / KS / PSI 才是核心，附 3 个必懂指标速记

- 把业务 SLO 绑进 release manifest，让上线决策"业务指标也得过"

TECH $\neq$ BIZ

技术指标好 $\neq$ 业务好——评测体系必须有一条"业务漏斗"做翻译

$F=0.9$ 是技术指标；"一次解决率 50% $\rightarrow$ 65%"是业务指标，中间隔着好几环



关键判断

每一环都有"漏斗损失"。如果技术指标涨了、业务指标却没动，一定是中间某一环卡住了——比如答案质量上去了（Quality $\uparrow$ ），但 UI 太难用、用户看不懂（Behavior 没变），工单照样没关。所以评测体系从第一天就要把这条漏斗画出来，追的是 end-to-end 的业务影响，不是只盯两端的数字自嗨。

FINANCE CUSTOMER SERVICE

金融客服域的 4 类核心业务 SLO——任何一类不达标，只能内部 Beta，不能 GA

业务方关心的不是 "AI 答得对不对"，是 "问题解决没、效率提没提、合规兜没兜住"

类别	关键指标	目标值	关联技术指标
自助 + 闭环	一次解决率 / 自助分流 / 工单闭环 / 升级率	>65% / >70% / >90% / <10%	Faithfulness + AR
效率 + 成本	AHT / TTFR / 每单成本 / 重复咨询率	<5min / <30s / <¥3 / <15%	Latency + Cost
质量 + 体验	CSAT / NPS / 投诉率 / 无用回答占比	>4.0 / >30 / <2% / <5%	AR + Citation
风险 + 合规	合规拦截率 / 误导率 / PII 泄露 / 引用覆盖	>99% / <0.1% / =0 / >95%	Adversarial + Audit

这 4 类合起来 = "金融客服 Copilot 上线门禁"。最后一行"风险+合规"是红线中的红线——前三类不达标顶多体验差，第四类不达标是监管事件。我的标准：质量、效率类可以"边上线边优化"，合规类必须"100% 达标才准 GA"，一例 PII 泄露就够你上新闻。技术指标和业务指标的对应关系（右列）要从设计期就 wire 死。

COMPLIANCE RED LINES

金融业的 4 条硬性合规指标——超线直接监管处罚 + 业务暂停

这 4 个不是“优化项”，是“红线”——踩了就不是扣分，是出局

合规拦截率 $\geq 99\%$

- “保证赚钱”“零风险”
- 等高风险话术
- 必须被 AI 主动拦
- 低于 99% = 罚款 + 停业务

高风险误导率 $< 0.1\%$

- 比拦截更严
- Agent 自己不能说违规话
- 大流量下 0.1%
- = 每天数百次违规

PII 泄露率 = 0

- 身份证 / 卡号 / 手机号
- 任何 PII 进答案
- = 监管事件
- 必须 100% 零事故

引用证据覆盖 $\geq 95\%$

- 每条专业建议
- 必须能反查依据
- 监管审计抽查时
- 这是合规底线

踩坑提醒

注意这 4 条和前面的质量指标本质不同：质量指标是“越高越好”的连续优化，合规指标是“过线/出局”的二值红线。别把它们混在一个看板里用同样的颜色——我的做法是合规指标单独一块、红色、带阈值告警，Pre-Release Gate 里强制校验，一条不过，整个发布卡死。

RISK MANAGEMENT METRICS

风控域评测完全不同——客服测"答得好不好"，风控测"判得对不对"

风控是离散决策，错一次直接对应资金损失，比客服评测严格 5 倍

类别	具体指标	看什么
离线模型效果	AUC / KS / Gini / Precision / Recall / PR-AUC	能不能区分好坏样本
在线业务效果	拦截损失 / 实际损失率 / 通过率 / 误杀成本 / 净收益	上线后的真实业务结果
运营 + 人审	告警量 / 命中率 / 人审吞吐 / 处理时长 / 积压	运营扛不扛得住
稳定性 + 漂移	PSI / 特征漂移 / 标签漂移 / 阈值敏感性	模型在不在悄悄劣化
合规 + 解释	原因码覆盖 / 解释一致性 / 公平性 / 审计可追溯	能不能解释 + 合规

风控这 5 类必须并行盯，不能只看 AUC。我见过团队 AUC 0.88 很得意，结果"在线业务效果"那栏一看——误杀成本高到把拦截收益全吃光，净收益是负的。模型再"准"，业务上亏钱也是失败。尤其"稳定性+漂移"那栏的 PSI，是风控模型的命门——它告诉你模型该不该重训了。

AUC / KS / PSI

风控 3 个最常用指标的快速理解——不懂就跟业务方对不上话

这 3 个是风控同学张口闭口的“黑话”，必须门清

AUC 区分能力

- 任取 1 好 1 坏
- 模型给坏的打分更高的概率
- $AUC=0.85 \rightarrow 85\%$ 概率排对
- > 0.75 可用 / > 0.85 优秀

KS 区分峰值

- 好坏累积分布最大差距
- $KS=0.4 \rightarrow$ 某阈值“抓 70% 坏”
- 同时误伤 30% 好”
- > 0.3 可用 / > 0.45 优秀

PSI 分布漂移

- 当前分布 vs 训练分布
- < 0.1 稳定 / $0.1-0.25$ 轻微
- > 0.25 严重漂移
- 是“该不该重训”最重要信号

关键判断

AUC 看“整体排序能力”，KS 看“最佳阈值处的区分峰值”，PSI 看“模型有没有过期”。前两个是上线前看的（模型好不好），PSI 是上线后天天看的（模型坏没坏）。三个一起，才是风控模型的完整体检。

BUSINESS SLO IN MANIFEST

把业务 SLO 也绑进 release manifest——上线决策的最后一道门

上线不再只看技术分，业务指标也得过；Canary 回滚直接挂业务红线

```
# release/manifests/rag-v2026.05.18-001.yaml
kind: RAGRelease
spec:
  # ...index / prompt / model / skill / eval
  business_slo:      # Week 11 ← 业务也必绑
    domain: finance_customer_service
    metrics:
      first_resolution_rate: {target: 0.65,
        current: 0.68, status: pass}
      self_service_rate:   {target: 0.70,
        current: 0.71, status: pass}
      cost_per_ticket:     {target: "<3.0",
        current: 2.4, status: pass}
      csat:                {target: 4.0,
        current: 4.2, status: pass}
    # 合规红线
    compliance_block_rate: {target: 0.99,
      current: 0.997, status: pass}
    pii_leak_rate:         {target: 0,
      current: 0, status: pass}
  rollout:
    auto_rollback_on:      # Canary 挂业务指标
    - {metric: pii_leak_rate, threshold: 0}  # 一例就回滚
    - {metric: first_resolution_rate,
      threshold: "<0.62"}
```

business_slo 段 · 上线决策不再只看技术分，业务指标也必须过——这是给 CEO 看的那一栏

每个指标带 target / current / status · 一眼看出哪个不达标

合规红线单独成组 · pii_leak / compliance_block, Pre-Release Gate 强制校验

auto_rollback_on 直接挂业务指标 · Canary 期间 PII 泄露一例就秒回滚——这是 Week 11 给 Week 14 治理的关键贡献

技术指标、业务指标、合规红线，全收进一份 manifest——这就是“评测驱动发布”的完整形态

INDUSTRY 2026

行业共识：AI 系统的"成功"必须用业务指标定义，技术指标只是手段

从"技术 KPI"转向"业务 OKR + 合规双指标"

来源	做法	启示
Google SRE	所有系统都有业务 SLO + 错误预算	"100% 可用"不是目标，"满足业务"才是
Anthropic AUP + Evals	把合规拦截做成产品级评测维度	能不能拦住合规风险是硬要求
Salesforce Einstein	AI 成功指标直接挂 Sales/Service KPI	AI 好不好 = 业务好不好
中国 央行 / 金监 / 证监	AI 客服/风控要"业务效果 + 合规"双指标	不能只做技术评测，要算法备案

记死

2026 年的事实 → AI 系统的"成功"，必须用业务指标来定义。技术评测是地基，但只有地基没有楼。尤其在金融，监管要的是"业务效果 + 合规"双达标——你 RAGAS 跑得再漂亮，合规拦截率不到 99%，一样上不了线。把业务 SLO 和合规红线绑进 manifest，是这一周送给 Week 14 治理最重的一份礼。

WEEK 11 MAP

Week 11 五节完整能力栈——一套"可量化、可门禁、可监管"的评测体系

从评测集，到业务 SLO，闭环了



关键判断

这 5 节连起来，就是一套完整的评测体系：有数据（Dataset）、有刻度（Metrics）、有可信的裁判（Judge）、有牙齿的门禁（Gate）、有业务和合规的落点（Biz）。Copilot 的"质量"，第一次从"我觉得还行"，变成了一个能写进 release、能卡住 PR、能交给监管看的工程指标。Week 12 起进入可观测——把评测从"事前事中"，扩展到"事后实时观察"。

WEEK 11 · THE END

你已经造好了一套完整的评测体系

从评测集资产化、到 RAGAS 量化、到 LLM 裁判、到 CI 门禁、再到业务 SLO——Copilot 的质量第一次从“经验判断”变成了工程指标

本周交付物（已 push 至 GitHub 仓库）

Eval Dataset v2.3

evals/v2.3.0.json

Ragas Runner

evals/run_ragas.py

Judge Prompts

evals/judges/*.j2

Calibrate Tool

evals/calibrate.py

AB Test

evals/ab_test.py

Eval Gate CI

.github/workflows/rag-eval-gate.yml

下周 → Week 12 · 全链路可观测性：OpenTelemetry / Tracing / Bad Case 复盘 / 故障定位——把 Agent 系统的故障定位，从“小时级”降到“分钟级”。评测让你知道“好不好”，可观测让你知道“出事了在哪、为什么”。